Sprint 14 : React Testing Foundations, Unit Tests, RTL, TDD/BDD, Assertions, and Test Doubles.

0. Where You Are Before Sprint 14 (Sprint 14 පටන් ගන්න කලින් ඔයා සිටිය යුතු තැන)

- **Concept Explanation (Sinhala):** Sprint 14 එක පටන් ගන්න කලින්, ඔයා React වල මූලික සිද්ධාන්ත සහ concepts ගැන හොඳ අවබෝධයක් ලබාගෙන ඉන්න ඕනේ. ඒ දේවල් තමයි:
  - React වලින් Components පාවිච්චි කරලා User Interfaces (UI) build කරන විදිහ.
  - JavaScript ඇතුළේ UI ලියන්න JSX පාවිච්චි කරන විදිහ.
  - Parent component එකක ඉඳන් Child component එකකට Props හරහා data pass කරන විදිහ.
  - වෙනත් JSX elements wrap කරන්න `children` prop එක පාවිච්චි කරන හැටි.
  - Data වල වෙනස්වීම් අනුව UI එකේ විවිධ කොටස් පෙන්වන්න Conditional Rendering පාවිච්චි කිරීම.
  - Lists render කරන්න `map()` function එක සහ React එකට items track කරන්න උදව් වෙන stable keys පාවිච්චි කිරීම.
  - Click events, typing events සහ form submissions වලට React responds කරන්නේ Event Handlers හරහා බව.
  - Forms වල values React state එකේ store කරන්න Controlled inputs පාවිච්චි කරන විදිහ.
  - වෙනස් වන data මතක තියාගන්න `useState` hook එක පාවිච්චි කිරීම සහ State updates හැමවිටම Immutable (වෙනස් කළ නොහැකි) ලෙස තබා ගැනීම.
  - සංකීර්ණ state actions එකතු කරලා සංවිධානය කරන්න `useReducer` පාවිච්චි කිරීම.
  - UI එක re-render කරන්නේ නැතිව DOM elements වලට access ගන්න හෝ values store කරන්න `useRef` පාවිච්චි කිරීම.
  - React render එකකට පස්සේ Side effects රන් කරන්න `useEffect` පාවිච්චි කිරීම සහ පරණ timers, listeners, subscriptions නවත්වන්න cleanup functions ලියන හැටි.
  - කෝඩ් එක නැවත පාවිච්චි කරන්න Custom Hooks හැදීම.
  - App-level එකේ (උදාහරණයක් විදිහට current user, theme, language වගේ) values share කරන්න Context API පාවිච්චි කිරීම.
  - Backend APIs වලින් data fetch කිරීම සහ Loading, Data, Error states හරහා async UI එකක් හැසිරවීම.
  - අනවශ්‍ය re-renders වළක්වලා performance වැඩි කරන්න `useMemo`, `useCallback`, සහ `React.memo` පාවිච්චි කිරීම.
  - URLs, React components එක්ක සම්බන්ධ කරන්න React Router පාවිච්චි කිරීම.
  - Styling කරන්න CSS frameworks සහ preprocessors පාවිච්චි කිරීම.
  - HTML, CSS, JavaScript, images, සහ fonts එකතු වෙලා browser එක ඇතුලේ pixels විදිහට render වෙන හැටි (Browser rendering).
  - Performance measure කරන්න විවිධ performance tools පාවිච්චි කිරීම.

  අපි මෙතෙක් ඉගෙන ගත්ත ප්‍රධානම ප්‍රශ්නය: අපි කොහොමද වැඩ කරන *React app* එකක් *build* කරන්නේ? **Sprint 14 ඉඳන් අපේ ප්‍රශ්නය වෙන්නේ:** අපි කෝඩ් එකක් වෙනස් කරාට පස්සේ අපේ *React app* එක තවමත් කලින් වගේම හරියට වැඩ කරනවා කියලා *check* කරගන්නේ කොහොමද? මෙන්න මේකට තමයි අපි **Testing** පාවිච්චි කරන්නේ.

1. Sprint 14 Goal (Sprint 14 හි ඉලක්කය)

- **Concept Explanation (Sinhala):** මේ Sprint එකේ ප්‍රධානම අරමුණ තමයි Testing වල මූලික පදනම (Testing Foundations) හරියටම වටහා ගන්න එක. මේක ඉවර වෙනකොට ඔයාට:
  - Testing වැදගත් වෙන්නේ ඇයි කියලා පැහැදිලි කරන්න.
  - Manual testing සහ Automated testing අතර වෙනස පැහැදිලි කරන්න.
  - SDLC (Software Development Life Cycle) එකේ පියවරවල් තේරුම් ගන්න.
  - සරල Unit tests ලියන්න.
  - මූලික test syntax (උදා: `test()`, `describe()`, `expect()`, සහ Matchers) තේරුම් ගන්න.
  - Test එකක් Pass වීම සහ Fail වීම කියන්නේ මොකක්ද කියලා වටහා ගන්න.
  - කුඩා utility functions වලට tests ලියන්න.
  - Assertions කියන්නේ මොනවද කියලා දැනගන්න.
  - TDD (Test-Driven Development) සහ BDD (Behavior-Driven Development) ගැන අවබෝධයක් ලබාගන්න.
  - Mocks, Stubs, සහ Fakes අතර වෙනස තේරුම් ගන්න.
  - React Testing Library (RTL) එකෙන් basic component testing කරන විදිහ ඉගෙන ගන්න.
  - Component එකක ඇසට පෙනෙන (visible behavior) දේවල් test කරන්න සහ මුල් කාලයේදීම implementation details (කෝඩ් එක ඇතුලේ ලියලා තියෙන විදිහ) test කිරීමෙන් වළකින්න ඉගෙන ගන්න.

2. Big Mental Model (ප්‍රධාන මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** අපි app එකක් හදනකොට, මේ වගේ වෙලාවල් වලදී bugs ඇති වෙන්න පුළුවන්:
  - අලුත් feature එකක් එකතු කරද්දී.
  - පරණ කෝඩ් එකක් වෙනස් කරද්දී.
  - Component එකක් refactor කරද්දී.
  - Dependency එකක් update කරද්දී.
  - එක බග් එකක් fix කරන්න ගිහින් නොදැනුවත්වම තවත් බග් එකක් ඇති වෙද්දී.

  Testing කියන්නේ මේ හැමදේකින්ම අපේ කෝඩ් එක ආරක්ෂා කරගන්න තියෙන safety check එකක්.

  Basic Testing Flow:

  [Code behavior]
  |
  ▼
  [Expected result]
  |
  ▼
  [Test checks the result]
  |
  ├──► [PASS] ── (behavior works)
  |
  └──► [FAIL] ── (something is wrong)

  Testing වලින් කරන්නේ අපේ කෝඩ් එක 100% perfect කියලා ඔප්පු කරන එක නෙමෙයි. Testing වලින් කරන්නේ අපේ app එකේ තියෙන වැදගත්ම හැසිරීම් (important behaviors) ආරක්ෂා කරගන්න එකයි. **Beginner Rule:** ඔයාගේ function එකකට හෝ component එකකට වැදගත් හැසිරීමක් තියෙනවා නම්, ඒකට පොඩි test එකක් ලියන්න.

3. What Is Testing? (Testing කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** සරලවම කිව්වොත් Testing කියන්නේ අපේ කෝඩ් එක අපි බලාපොරොත්තු වෙන විදිහටම (expected behavior) වැඩ කරනවද කියලා check කරන එකයි.
- **Code Block:**

```
expect(2 + 2).toBe(4);
```

- **Explanation:** මෙකෙන් කියන්නේ: "මම බලාපොරොත්තු වෙනවා 2 + 2 හි එකතුව 4 වෙයි කියලා." උත්තරේ 4 ආවොත් test එක pass වෙනවා. නැත්නම් fail වෙනවා.

4. Why Testing Matters (Testing වැදගත් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** Tests නැතිව වැඩ කරද්දී අපිට හැමතිස්සෙම app එක manually check කරන්න වෙනවා. Browser එක open කරන්න, button එකක් click කරන්න, ටයිප් කරන්න, submit කරලා බලන්න. මේක ලොකු app එකක හැමදාම කරන්න ගියොත් සෑහෙන වෙලාවක් යනවා. Testing වැදගත් වෙන්නේ මේ නිසයි:
  - Bugs කලින්ම හොයාගන්න පුළුවන්.
  - අලුත් කෝඩ් එකතු කරද්දී පරණ features කැඩෙන්නේ නැහැ.
  - බයක් නැතුව කෝඩ් එක refactor කරන්න පුළුවන්.
  - කෝඩ් එක වැඩ කරන්න ඕනේ කොහොමද කියලා ලියවුණු ලිඛිත සාක්ෂියක් (documentation) වෙනවා.
  - මගහැරෙන්න පුළුවන් edge cases ලේසියෙන්ම check කරගන්න පුළුවන්.

5. Manual Testing (මැනුවල් ටෙස්ටිං)

- **Concept Explanation (Sinhala):** Manual testing කියන්නේ මනුස්සයෙක් විසින්ම තමන්ගේ ඇසින් සහ අතින් app එක පාවිච්චි කරලා බලලා check කරන එකයි.

  Manual Testing Flow:

  [Open App]
  |
  ▼
  [Click Login]
  |
  ▼
  [Type email]
  |
  ▼
  [Type password]
  |
  ▼
  [Submit form]
  |
  ▼
  [Check dashboard appears]

  මෙය තවමත් ප්‍රයෝජනවත් වෙනවා visual layout එක, සැබෑ browser එකක හැසිරීම සහ user experience එක බලාගන්න. හැබැයි මේකේ ප්‍රධාන ගැටලු කිහිපයක් තියෙනවා:
  - **Slow:** හැමදාම එකම clicks ටික ආයෙ ආයෙ කරන්න වෙනවා (උදාහරණයක් විදිහට login එක හැමදාම අතින් test කරන්න වීම).
  - **Easy to forget:** පරණ features (උදා: password reset) test කරන්න අමතක වෙන්න පුළුවන්.
  - **Inconsistent:** හැම සැරේම එකම විදිහට ටයිප් කරලා test කරන්නේ නැති වෙන්න පුළුවන්.
  - **Weak for hidden logic:** ඇසට පෙනෙන UI එකට යටින් තියෙන සංකීර්ණ logic (උදා: cart total එක ගණන් හදන හැටි) manually check කරන්න අමාරුයි.

6. Automated Testing (ඔටෝමේටඩ් ටෙස්ටිං)

- **Concept Explanation (Sinhala):** Automated testing කියන්නේ වෙනත් කෝඩ් එකක් ලියලා අපේ ප්‍රධාන කෝඩ් එක හරියාකාරව වැඩ කරනවද කියලා check කරගන්න එකයි.
- **Code Block:**

```
expect(formatName("react")).toBe("React");
```

- **Explanation:** මෙකෙන් `formatName("react")` කියන function එකෙන් ලැබෙන output එක `"React"` ද කියලා පරීක්ෂා කරනවා. මෙම tests නැවත නැවත ඉතා වේගයෙන් රන් කරන්න පුළුවන් නිසා **Regressions** (කලින් වැඩ කරපු දෙයක් අලුත් වෙනස්කම් නිසා කැඩී යාම) වළක්වා ගන්න ලොකු උදව්වක් වෙනවා.

7. Manual Testing vs Automated Testing (සංසන්දනය)

- **Concept Explanation (Sinhala):** - **Manual Testing:** කරන්නේ පුද්ගලයෙක් විසින්. වේගය අඩුයි. නැවත නැවත කිරීම වෙහෙසකරයි. User Experience සහ visual checks වලට වඩාත් සුදුසුයි. Edge cases අමතක වෙන්න තියෙන ඉඩ වැඩියි.
  - **Automated Testing:** කරන්නේ කෝඩ් එකකින්. වේගය ඉතා වැඩියි. නැවත නැවත කිරීම හරිම පහසුයි. Logic සහ repeatable behavior වලට වඩාත් සුදුසුයි. හැබැයි මුලින්ම test code එක ලියන්න කාලය වැය කරන්න සිද්ධ වෙනවා. සාමාන්‍යයෙන් සාර්ථක app එකකට මේ ක්‍රම දෙකම එකතුවක් අවශ්‍ය වෙනවා.

8. SDLC: Software Development Life Cycle (මෘදුකාංග සංවර්ධන ජීවන චක්‍රය)

- **Concept Explanation (Sinhala):** SDLC එකෙන් විස්තර කරන්නේ software එකක් හදනකොට පසුකරන ප්‍රධාන පියවරවල් ටිකයි.
  1. **Planning (සැලසුම් කිරීම):** මොකක්ද හදන්න ඕනේ කියලා තීරණය කිරීම. (උදා: අපිට signup form එකක් ඕනේ).
  2. **Design (නිර්මාණය):** ඒක වැඩ කරන්න ඕනේ සහ පේන්න ඕනේ කොහොමද කියලා තීරණය කිරීම. (උදා: form එකට name, email, password, සහ submit button එකක් තියෙන්න ඕනේ).
  3. **Implementation (කෝඩ් කිරීම):** කෝඩ් එක ලිවීම. (උදා: `SignupForm.jsx` එක build කිරීම).
  4. **Testing (පරීක්ෂා කිරීම):** කෝඩ් එක හරියටම වැඩ කරනවද කියලා බලන එක. (උදා: name field එක හිස්ව තිබ්බොත් error එකක් එනවද කියලා බලන එක).
  5. **Deployment (නිකුත් කිරීම):** App එක users ලට පාවිච්චි කරන්න live දාන එක.
  6. **Maintenance (නඩත්තු කිරීම):** පස්සේ එන bugs fix කිරීම සහ update කිරීම සිදු කිරීම.

9. Testing in a React Workflow (React Workflow එකක් තුළ testing කරන හැටි)

- **Concept Explanation (Sinhala):** React app එකක විවිධ කොටස් අපිට විවිධ විදිහට test කරන්න පුළුවන්:
  - **Utility function:** Return වෙන අගය (return value) check කිරීම.
  - **Button component:** Button එකේ පෙනෙන text එක නිවැරදිදැයි check කිරීම.
  - **Alert component:** Error message එකක් හරියට screen එකේ පෙන්වනවද කියලා check කිරීම.
  - **Form component:** Input fields, labels සහ validation messages හරියට තියෙනවද කියලා check කිරීම.
  - **Refactor:** කෝඩ් එක වෙනස් කරන්න කලින් සහ පස්සේ පරණ tests රන් කරලා බැලීම.
  - **Bug fix:** බග් එකක් ආවම, ඒ බග් එකට අදාළව test එකක් ලියලා, ආයේ කවදාවත් ඒ බග් එක නොඑන විදිහට කෝඩ් එක හැදීම.

10. Unit Testing (යුනිට් ටෙස්ටිං)

- **Concept Explanation (Sinhala):** Unit test එකක් මඟින් කරන්නේ මුළු app එකම නෙමෙයි, කෝඩ් එකේ තියෙන ඉතාම කුඩා කොටසක් (Isolated unit) තනිවම ගෙන පරීක්ෂා කිරීමයි. මේක තනි function එකක් හෝ කුඩා component එකක් වෙන්න පුළුවන්.
- **Code Block:**

```
// add.js
export function add(a, b) {
  return a + b;
}

// add.test.js
import { add } from "./add";

test("adds two numbers", () => {
  expect(add(2, 3)).toBe(5);
});
```

- **Explanation:** මෙහි `add(2, 3)` යන්නෙන් ලැබෙන අගය 5 විය යුතු බවට තනි unit එකක් ලෙස පරීක්ෂා කර තිබේ.

11. Pure Functions Are Good First Test Targets (පළමුවෙන්ම ටෙස්ට් කරන්න හොඳ Pure Functions)

- **Concept Explanation (Sinhala):** Pure Function එකක් කියන්නේ එකම input එකට හැමතිස්සෙම එකම output එක දෙන, වෙනත් කිසිම බාහිර data එකක් වෙනස් නොකරන (no side effects) function එකකටයි. මේවා test කරන්න හරිම ලේසියි. මොකද browser එකක් හෝ React rendering අවශ්‍ය වෙන්නේ නැහැ.
- **Code Block:**

```
export function double(number) {
  return number * 2;
}

// Test code
test("doubles a number", () => {
  expect(double(4)).toBe(8);
});
```

12. Basic Test Syntax (මූලික Test රටාව)

- **Concept Explanation (Sinhala):** සාමාන්‍යයෙන් ලියන සරලම test එකකම මේ වගේ හැඩයක් තියෙනවා:
  - `test()` : තනි test case එකක් හදනවා.
  - `"description"` : Test එකෙන් කරන්නේ මොකක්ද කියලා ලියන විස්තරය.
  - `() => {}` : Test කෝඩ් එක ක්‍රියාත්මක වන function එක.
  - `expect(...)` : අපිට ලැබුණු සැබෑ අගය (actual value).
  - `.toBe(...)` : අපිට ලැබෙන්න ඕනේ කියලා බලාපොරොත්තු වෙන අගය (expected value).

13. test()

- **Concept Explanation (Sinhala):** `test()` function එකෙන් කරන්නේ තනි test එකක් define කරන එකයි. පළමු argument එක විදිහට පැහැදිලි විස්තරයක් (description) සහ දෙවන argument එක විදිහට test එක රන් වෙන function එක ලබා දෙනවා.
- **Code Block:**

```
test("returns 0 for an empty cart", () => {});
test("shows an error message when name is empty", () => {});
```

- **Explanation:** හැමවිටම "works" හෝ "test 1" වැනි අපැහැදිලි නම් නැතිව, සිදුවන දේ පැහැදිලිව විස්තර කරන නමක් (good description) test එකට තබන්න.

14. describe()

- **Concept Explanation (Sinhala):** එකිනෙකට සම්බන්ධ tests කිහිපයක් එක Group එකකට එකතු කරන්න (group related tests) `describe()` පාවිච්චි කරනවා.
- **Code Block:**

```
describe("formatName", () => {
  test("capitalizes lowercase input", () => {
    expect(formatName("react")).toBe("React");
  });

  test("returns empty string for empty input", () => {
    expect(formatName("")).toBe("");
  });
});
```

15. expect()

- **Concept Explanation (Sinhala):** Assertion එකක් පටන් ගන්නේ `expect()` එකෙන්. Assertion එකක් කියන්නේ අපේ කෝඩ් එකේ ප්‍රතිඵලය සහතික කරන්න කියලා පරීක්ෂා කරන එකටයි. `expect()` ඇතුළට අපි දෙන්නේ අපේ function එකෙන් ලැබුණු සැබෑ අගයයි (actual value).

16. Matchers (මැචර්ස්)

- **Concept Explanation (Sinhala):** `expect()` එකට පස්සේ අපි ලියන method එකට තමයි Matcher එකක් කියන්නේ. මේවායින් තමයි සැබෑ අගය සහ බලාපොරොත්තු වන අගය සංසන්දනය කරන්නේ.
  - `toBe(value)` : Exact values (Numbers, Strings, Booleans, null, undefined) සංසන්දනය කරන්න.
  - `toEqual(value)` : Objects සහ Arrays ඇතුළේ තියෙන data සමානද කියලා බලන්න පාවිච්චි කරනවා (Object references වෙනස් වුණත් අගයන් සමාන නම් pass වේ).
  - `toContain(item)` : Array එකක් ඇතුළේ යම්කිසි අගයක් අඩංගුද කියා බැලීමට.
  - `toBeInTheDocument()` : RTL වලදී, යම් UI Element එකක් screen එකේ (DOM) තියෙනවද කියා බැලීමට.
  - `toHaveTextContent(text)` : UI Element එකක් ඇතුළත පෙනෙන text එක check කිරීමට.

17. Passing Tests (සමත් වන ටෙස්ට්)

- **Concept Explanation (Sinhala):** අපිට ලැබුණු සැබෑ ප්‍රතිඵලය (actual result) සහ අපි බලාපොරොත්තු වූ ප්‍රතිඵලය (expected result) 100% ක් සමාන වූ විට test එකක් pass වෙනවා.

18. Failing Tests (අසමත් වන ටෙස්ට්)

- **Concept Explanation (Sinhala):** කෝඩ් එකේ බග් එකක් නිසා හෝ වැරදීමක් නිසා බලාපොරොත්තු වූ අගය නොලැබී ගියොත් test එක fail වෙනවා. Failing tests මඟින් අපිට ලැබුණු වැරදි අගය (received) සහ ලැබෙන්න තිබ්බ නිවැරදි අගය (expected) මොකක්ද කියලා පෙන්වලා දෙනවා.

19. Example: formatName() Utility Function

- **Concept Explanation (Sinhala):** අපි string එකක පළමු අකුර uppercase කරන සරල utility function එකක් ලියමු. හිස් අගයක් ආවොත් හිස් string එකක් return කරන්න ඕනේ.
- **Code Block:**

```
// src/utils/formatName.js
export function formatName(str = "") {
  if (!str) return "";
  return str.charAt(0).toUpperCase() + str.slice(1);
}
```

20. Test: formatName()

- **Concept Explanation (Sinhala):** ඉහත ලියපු `formatName` function එකට සාමාන්‍ය අවස්ථා (happy path) මෙන්ම edge cases (හිස් අගයක්) වලදී නිවැරදිව වැඩ කරනවද කියලා බලන්න ලියන test file එක මෙන්න.
- **Code Block:**

```
// src/utils/formatName.test.js
import { describe, expect, test } from "vitest";
import { formatName } from "./formatName";

describe("formatName", () => {
  test("capitalizes lowercase input", () => {
    expect(formatName("react")).toBe("React");
  });

  test("returns empty string for no input", () => {
    expect(formatName()).toBe("");
  });

  test("keeps already-capitalized input correct", () => {
    expect(formatName("React")).toBe("React");
  });
});
```

21. Edge Cases (එජ් කේසස්)

- **Concept Explanation (Sinhala):** Edge case එකක් කියන්නේ සාමාන්‍යයෙන් කලාතුරකින් සිදුවිය හැකි, බලාපොරොත්තු නොවන අසාමාන්‍ය input එකකටයි.
  - සාමාන්‍ය අවස්ථාව (Normal/Happy path): `formatName("react")`
  - Edge cases: `formatName("")`, `formatName(undefined)`, `formatName("a")` Bugs හැංගිලා තියෙන්නේ මේ වගේ edge cases අස්සේ නිසා මේවා test කිරීම හරිම වැදගත්.

22. Happy Path (හැපි පාත්)

- **Concept Explanation (Sinhala):** හැපි පාත් එකක් කියන්නේ කිසිදු බාධාවක් හෝ වැරදීමක් නැතිව සාමාන්‍ය විදිහටම app එක වැඩ කරන සරලම සාර්ථක මාර්ගයයි. (උදා: Signup form එකට නිවැරදි විස්තර ටික දාලා submit කරලා සාර්ථකව signup වෙන එක).

23. Test Files (ටෙස්ට් ෆයිල්ස් සකසන ආකාරය)

- **Concept Explanation (Sinhala):** සාමාන්‍යයෙන් test file එකක් හදන්නේ ප්‍රධාන කෝඩ් එක තියෙන ෆයිල් එකේ නමට පස්සේ `.test.` කෑල්ලක් දාලා. ඉගෙන ගන්න කාලේ ලේසි වෙන්න ප්‍රධාන ෆයිල් එක තියෙන ෆෝල්ඩර් එකේම ටෙස්ට් ෆයිල් එකත් තියන්න.
  - `formatName.js` -> `formatName.test.js`
  - `Button.jsx` -> `Button.test.jsx`

24. Vitest Beginner Context (Vitest හැඳින්වීම)

- **Concept Explanation (Sinhala):** Vite වලින් හදන React apps වලදී අපි බහුලවම පාවිච්චි කරන්නේ **Vitest** කියන testing framework එකයි. React components test කරන්න `@testing-library/react` සහ DOM matchers පාවිච්චි කරන්න `@testing-library/jest-dom` අවශ්‍ය වෙනවා.

25. Cart Utility Example

- **Concept Explanation (Sinhala):** Cart එකේ ඇති බඩු බාහිරාදියෙහි මුළු එකතුව (Total Price) සෙවීමට ලියන ලද utility function එකක් මෙන්න.
- **Code Block:**

```
// src/utils/calculateCartTotal.js
export function calculateCartTotal(items = []) {
  return items.reduce((total, item) => {
    return total + item.price * item.quantity;
  }, 0);
}
```

26. Test: calculateCartTotal()

- **Concept Explanation (Sinhala):** හිස් cart එකකට, එක අයිටම් එකකට සහ අයිටම් කිහිපයකට ඉහත function එක හරියට වැඩ කරනවද කියලා බලන්න ලියන tests මෙන්න.
- **Code Block:**

```
// src/utils/calculateCartTotal.test.js
import { describe, expect, test } from "vitest";
import { calculateCartTotal } from "./calculateCartTotal";

describe("calculateCartTotal", () => {
  test("returns 0 for an empty cart", () => {
    expect(calculateCartTotal([])).toBe(0);
  });

  test("calculates total for one item", () => {
    expect(
      calculateCartTotal([{ price: 10, quantity: 2 }])
    ).toBe(20);
  });

  test("calculates total for multiple items", () => {
    expect(
      calculateCartTotal([
        { price: 10, quantity: 2 },
        { price: 5, quantity: 3 },
      ])
    ).toBe(35);
  });
});
```

27. Assertions (ඇසර්ෂන්ස්)

- **Concept Explanation (Sinhala):** Assertion එකක් කියන්නේ test එකක් ඇතුළේ කරන එක් පරීක්ෂාවක්. එක් test එකක් ඇතුළේ අපිට assertions එකක් හෝ කිහිපයක් කරන්න පුළුවන්. හැබැයි හැමතිස්සෙම එක test එකක් ඇතුළේ එක පැහැදිලි හැසිරීමක් (one clear behavior) විතරක් check කරන්න පුරුදු වෙන්න.

28. TDD: Test-Driven Development (ටෙස්ට් ඩ්‍රිවන් ඩෙවලොප්මන්ට්)

- **Concept Explanation (Sinhala):** TDD කියන්නේ කෝඩ් එක ලියන්න කලින්, ඒ කෝඩ් එකෙන් ලැබෙන්න ඕනේ ප්‍රතිඵලයට අදාළව test එක ලියලා ඉන්න එකටයි. මෙහි ප්‍රධාන පියවර 3 ක් තියෙනවා:
  1. **Red:** මුලින්ම test එක ලියනවා. කෝඩ් එක තවම ලියලා නැති නිසා test එක fail වෙනවා (Red වෙනවා).
  2. **Green:** Test එක pass වෙන්න අවශ්‍ය අවම කෝඩ් ප්‍රමාණය ලියනවා (Green වෙනවා).
  3. **Refactor:** කෝඩ් එකේ හැසිරීම වෙනස් නොකර කෝඩ් එක ලස්සනට සහ කාර්යක්ෂමව සකස් කරනවා (Refactor).

29. TDD Example

- **Concept Explanation (Sinhala):** අපි හිතමු `calculateCartTotal` ලියන්න කලින් අපි test එක ලියනවා. මුලින්ම test එක රන් කරද්දී අපිට error එකක් එනවා function එක නැති නිසා. ඊටපස්සේ අපි අවම කෝඩ් එක ලියලා ඒක green කරගන්නවා.
- **Code Block:**

```
// 1. මුලින්ම ලියන ටෙස්ට් එක (fails initially)
import { expect, test } from "vitest";
import { calculateCartTotal } from "./calculateCartTotal";

test("adds item prices", () => {
  expect(
    calculateCartTotal([
      { price: 10, quantity: 2 },
      { price: 5, quantity: 1 },
    ])
  ).toBe(25);
});
```

30. BDD: Behavior-Driven Development (බිහේවියර් ඩ්‍රිවන් ඩෙවලොප්මන්ට්)

- **Concept Explanation (Sinhala):** BDD වලදී අවධානය යොමු කරන්නේ තාක්ෂණික කෝඩ් එකට වඩා පරිශීලකයාගේ (user) හැසිරීම සහ ව්‍යාපාරික අවශ්‍යතා (business view) මතයි. මෙයට විශේෂ රටාවක් තියෙනවා:
  - **Given (යම්කිසි ආරම්භක අවස්ථාවකදී):** Given the user enters a valid email and password
  - **When (යමක් සිදු වූ විට):** When the user submits the login form
  - **Then (සිදුවිය යුතු අවසාන ප්‍රතිඵලය):** Then the dashboard should appear

31. BDD Example

- **Concept Explanation (Sinhala):** BDD රටාවට අනුව නිතන ක්‍රම කිහිපයක්:
  - *Cart validation:* Given the cart has two products, When the cart total is calculated, Then the total should include price times quantity for each product.
  - *Form validation:* Given the name field is empty, When the user submits the form, Then an error message should appear.

32. TDD vs BDD (සංසන්දනය)

- **Concept Explanation (Sinhala):** - **TDD:** අවධානය යොමු කරන්නේ "මේ function එකෙන් මොකක්ද return වෙන්න ඕනේ?" කියන තාක්ෂණික පැත්තටයි. (Technical test cases).
  - **BDD:** අවධානය යොමු කරන්නේ "User ට මොනවගේ අත්දැකීමක්ද ලැබෙන්න ඕනේ?" කියන හැසිරීම් පැත්තටයි. (Given/When/Then).

33. Mocks, Stubs, and Fakes (ටෙස්ට් ඩබල්ස්)

- **Concept Explanation (Sinhala):** ටෙස්ට් කරනකොට බාහිර සේවාවන් (Real APIs, Databases, Payment Gateways) පාවිච්චි කරන්න ගියොත් අපේ tests slow වෙනවා වගේම අපිට පාලනය කරන්න බැරි විදිහට fail වෙන්න පුළුවන්. ඒ නිසා ඒ වෙනුවට අපි බොරු ආදේශක (controlled replacements) පාවිච්චි කරනවා.

34. Mock (මොක්)

- **Concept Explanation (Sinhala):** Mock එකක් කියන්නේ සැබෑ දත්ත වෙනුවට පාවිච්චි කරන බොරු data හෝ object එකක්. මේකෙන් අපිට අවශ්‍ය විදිහට data ලැබෙනවා කියලා බොරුවට පෙන්වන්න පුළුවන්.
- **Code Block:**

```
const mockPosts = [
  { id: 1, title: "Learn React" },
  { id: 2, title: "Write Tests" },
];
```

35. Stub (ස්ටබ්)

- **Concept Explanation (Sinhala):** Stub එකක් කියන්නේ හැම තිස්සෙම කලින් තීරණය කරපු එකම ස්ථාවර උත්තරයක් (predefined/fixed answer) දෙන සරල function එකක් හෝ ක්‍රමවේදයක්.
- **Code Block:**

```
function getUserStub() {
  return { id: 1, name: "Sangeeth" };
}
```

36. Fake (ෆේක්)

- **Concept Explanation (Sinhala):** Fake එකක් කියන්නේ සැබෑ සේවාව වගේම වැඩ කරන, හැබැයි ඉතාම සරල ක්‍රමයකට හදපු ආදේශකයක් (working but simplified replacement). (උදාහරණයක් විදිහට සැබෑ Database එකක් වෙනුවට array එකක් ඇතුළේ data තියාගන්නා සරල fake database එකක්).
- **Code Block:**

```
const fakeDb = {
  users: [],
  addUser(user) {
    this.users.push(user);
  },
  getUsers() {
    return this.users;
  },
};
```

37. Mock vs Stub vs Fake (කෙටි මතක සටහන)

- **Concept Explanation (Sinhala):** - **Mock:** Pretend object / data (බොරුවට පෙන්වන data).
  - **Stub:** Fixed answer (හැමවෙලේම දෙන ස්ථාවර උත්තරය).
  - **Fake:** Simple working version (ඇත්ත එක වගේ වැඩ කරන සරල එකක්).

38. Mock API Data Practice

- **Concept Explanation (Sinhala):** API එකකින් posts ටිකක් අරන් ඒකෙන් titles විතරක් extract කරගන්නා function එකක් සහ ඒක mock data දාලා test කරන හැටි මෙන්න.
- **Code Block:**

```
// src/utils/getPostTitles.js
export function getPostTitles(posts = []) {
  return posts.map((post) => post.title);
}

// src/utils/getPostTitles.test.js
import { expect, test } from "vitest";
import { getPostTitles } from "./getPostTitles";

test("extracts titles from mocked posts", () => {
  const mockPosts = [
    { id: 1, title: "Learn React" },
    { id: 2, title: "Write Tests" },
  ];

  expect(getPostTitles(mockPosts)).toEqual([
    "Learn React",
    "Write Tests",
  ]);
});
```

39. React Testing Library (RTL - රියැක්ට් ටෙස්ටිං ලයිබ්‍රරි)

- **Concept Explanation (Sinhala):** RTL මඟින් කරන්නේ සැබෑ පරිශීලකයෙක් (user) අපේ React component එක දිහා බලලා පාවිච්චි කරන විදිහටම (as a user) test කරන්න ඉඩ සලසා දීමයි.
  - හොඳ පුරුද්ද: ඇසට පෙනෙන text, buttons, links, labels, inputs test කිරීම.
  - නරක පුරුද්ද: Component එකේ ඇතුලාන්තයේ තියෙන state එකේ නම `isOpen` ද කියලා check කිරීම (Implementation details).

40. render (රෙන්ඩර්)

- **Concept Explanation (Sinhala):** `render()` function එකෙන් කරන්නේ අපි ලියපු React component එක ටෙස්ට් පරිසරය ඇතුළේ තාවකාලිකව රන් කරලා පෙන්වන එකයි.
- **Code Block:**

```
render(<Button label="Submit" onClick={() => {}} />);
```

41. screen (ස්ක්‍රීන්)

- **Concept Explanation (Sinhala):** `render()` කරපු component එක ඇතුළේ තියෙන දේවල් (text, buttons, inputs) අපිට හොයාගන්න උදව් කරන්නේ `screen` කියන object එකයි. (උදා: `screen.getByText("Submit")`).

42. Common React Testing Library Queries (පොදු RTL විමසුම්)

- **Concept Explanation (Sinhala):** - `getByText()` : Screen එකේ පෙනෙන්න තියෙන text එකක් හොයාගන්න පාවිච්චි කරයි.
  - `getByRole()` : Element එකක තියෙන HTML role එකෙන් (button, link, heading, alert) හොයාගන්න පාවිච්චි කරයි. මේක accessibility වලටත් ගොඩක් හොඳයි.
  - `getByLabelText()` : `<label>` එකකට සම්බන්ධ කරලා තියෙන input field එක හොයාගන්න පාවිච්චි කරයි.
  - `queryByRole()` : යම්කිසි element එකක් screen එකේ නොතිබිය යුතුයි කියා පරීක්ෂා කිරීමට මෙය පාවිච්චි කරයි (getByRole පාවිච්චි කළහොත් element එක නැතිවිට test එක crash වේ).

43. Component Test Example: Button

- **Concept Explanation (Sinhala):** සරල Button component එකක් සහ එය සාර්ථකව render වෙනවද කියා බලන test එක මෙන්න.
- **Code Block:**

```
// src/components/Button.jsx
export default function Button({ label, onClick }) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}

// src/components/Button.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Button from "./Button";

test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);
  expect(screen.getByRole("button", { name: "Submit"
})).toBeInTheDocument();
});
```

44. Component Test Example: Alert

- **Concept Explanation (Sinhala):** Conditional rendering සහිත Alert component එකක් පරීක්ෂා කිරීම. මෙහි message එකක් නැතිනම් alert එක පෙන්වන්නේ නැහැ.
- **Code Block:**

```
// src/components/Alert.jsx
export default function Alert({ type = "info", message }) {
  if (!message) return null;
  return (
    <div role="alert">
      {type}: {message}
    </div>
  );
}

// src/components/Alert.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Alert from "./Alert";

test("shows an alert message", () => {
  render(<Alert type="error" message="Name is required" />);
  expect(screen.getByRole("alert")).toHaveTextContent(
    "error: Name is required"
  );
});

test("renders nothing when message is missing", () => {
  render(<Alert />);
  expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

45. Simple Form Component Test Example

- **Concept Explanation (Sinhala):** Input field එකක් සහ Submit button එකක් සහිත සරල Form එකක් render වන ආකාරය test කිරීම.
- **Code Block:**

```
// src/components/SignupForm.jsx
export default function SignupForm() {
  return (
    <form>
      <label htmlFor="name">Name</label>
      <input id="name" type="text" />
      <button type="submit">Sign up</button>
    </form>
  );
}

// src/components/SignupForm.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import SignupForm from "./SignupForm";

test("renders name input and submit button", () => {
  render(<SignupForm />);
  expect(screen.getByLabelText("Name")).toBeInTheDocument();
  expect(screen.getByRole("button", { name: "Sign up"
})).toBeInTheDocument();
});
```

46. Testing Conditional Text

- **Concept Explanation (Sinhala):** User ලොග් වෙලා ඉන්නවද නැද්ද යන්න මත වෙනස් වන පණිවිඩයක් නිවැරදිව පෙන්වන්නේදැයි පරීක්ෂා කිරීම.
- **Code Block:**

```
// src/components/StatusMessage.jsx
export default function StatusMessage({ isLoggedIn }) {
  return (
    <p>{isLoggedIn ? "Welcome back" : "Please log in"}</p>
  );
}

// src/components/StatusMessage.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import StatusMessage from "./StatusMessage";

test("shows welcome message when logged in", () => {
  render(<StatusMessage isLoggedIn={true} />);
  expect(screen.getByText("Welcome back")).toBeInTheDocument();
});

test("shows login message when logged out", () => {
  render(<StatusMessage isLoggedIn={false} />);
  expect(screen.getByText("Please log in")).toBeInTheDocument();
});
```

47. Testing Props

- **Concept Explanation (Sinhala):** Component එකකට props pass කරාම ඒ props වල තියෙන values නිවැරදිව UI එකේ දර්ශනය වෙනවද කියලා බලන හැටි.
- **Code Block:**

```
// src/components/UserCard.jsx
export default function UserCard({ name, role }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
  );
}

// src/components/UserCard.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import UserCard from "./UserCard";

test("shows user name and role", () => {
  render(<UserCard name="Ano" role="Frontend Developer" />);
  expect(screen.getByText("Ano")).toBeInTheDocument();
  expect(screen.getByText("Frontend Developer")).toBeInTheDocument();
});
```

48. What Not to Test Yet (දැනටමත් ටෙස්ට් කරන්න යන්න එපා කියන දේවල්)

- **Concept Explanation (Sinhala):** ආරම්භකයෙක් විදිහට ඔයා මේ වගේ සංකීර්ණ දේවල් මුලින්ම test කරන්න යන්න එපා:
  - සංකීර්ණ API mocking.
  - සම්පූර්ණ authentication flows.
  - සම්පූර්ණ checkout flows (මුදල් ගෙවීම්).
  - React ඇතුළත තියෙන state එක කෙලින්ම test කිරීම.
  - හැම තැනටම snapshot tests දැමීම.
  - ලොකු End-to-End (E2E) frameworks පාවිච්චි කිරීම. නැවතත් මතක තබා ගන්න: **කුඩා pure utility functions සහ ඇසට පෙනෙන UI behavior විතරක් මුලින්ම test කරන්න.**

49. Implementation Details (කෝඩ් එක ලියා ඇති ආකාරය)

- **Concept Explanation (Sinhala):** Implementation details කියන්නේ user ට සෘජුවම පේන්නේ නැති, කෝඩ් එක ඇතුලේ අපි දාලා තියෙන logic සහ variables වල නම්.
  - **නරක ක්‍රමය:** `expect(component.state.isOpen).toBe(true);` (isOpen කියන state එකේ නම වෙනස් කරපු ගමන් මේ test එක fail වෙනවා, හැබැයි UI එක වැඩ කරන්න පුළුවන්).
  - **හොඳ ක්‍රමය:** `expect(screen.getByText("Menu")).toBeInTheDocument();` (User ට පෙනෙන Menu එක පේන්න තියෙන එකයි).

50. Tests as Documentation (ලේඛනගත කිරීමක් ලෙස ටෙස්ට්)

- **Concept Explanation (Sinhala):** හොඳින් ලියන ලද test එකක නම කියවපු ගමන්, ඒ කෝඩ් එකෙන් මොකක්ද වෙන්න ඕනේ කියලා වෙනත් developer කෙනෙකුට ලේසියෙන්ම තේරුම් ගන්න පුළුවන්. ඒ නිසා tests කියන්නේ documentation එකක් තමයි.

51. Test-First Thinking Without Full TDD (සම්පූර්ණ TDD නැතිවත් ටෙස්ට් මුලින් සිතීම)

- **Concept Explanation (Sinhala):** හැමතිස්සෙම කෝඩ් එක ලියන්න කලින් මේ ප්‍රශ්න ටික තමන්ගෙන්ම අහන්න:
  - මේ function එකේ input එක මොකක්ද?
  - බලාපොරොත්තු වෙන output එක මොකක්ද?
  - User ට මොකක්ද පේන්න ඕනේ?
  - අගයක් හිස් ආවොත් මොකද වෙන්නේ?

52. Common Beginner Testing Mistakes (ආරම්භකයින් කරන පොදු වැරදි)

- **Concept Explanation (Sinhala):** - **Bad Test Names:** "works" වගේ අපැහැදිලි නම් දැමීම.
  - **Testing Too Much at Once:** එකම test එකක් ඇතුළේ මුළු app එකම test කරන්න යාම.
  - **Testing Implementation Details:** UI එක වෙනුවට ඇතුළේ තියෙන state variables පරීක්ෂා කිරීම.
  - **Forgetting Edge Cases:** හිස් අගයන් (empty values) පරීක්ෂා කරන්න අමතක වීම.
  - **Using Real API Data:** Mock data වෙනුවට ඇත්තම API data පාවිච්චි කරන්න යාම.

53. What Makes a Good Beginner Test? (හොඳ Beginner Test එකක ලක්ෂණ)

- **Concept Explanation (Sinhala):** - කුඩා වීම.
  - පැහැදිලි වීම.
  - එක අවස්ථාවක් පමණක් පරීක්ෂා කිරීම.
  - හොඳ නමක් තිබීම.
  - බාහිර දේවල් මත යැපෙන්නේ නැති වීම (predictable).
  - Fail වුණහම හේතුව ලේසියෙන් හොයාගත හැකි වීම.

54. Sprint 14 Practice Block (සරල පුහුණු අභ්‍යාස)

- **Concept Explanation (Sinhala):** - **Drill 1:** Basic test එකක් ලියා එය සරලව පැහැදිලි කරන්න.
  - **Drill 2:** "works" හෝ "button" වැනි වැරදි test names වෙනුවට "returns empty string when no name is provided" සහ "shows the submit button" වැනි පැහැදිලි නම් යොදන්න.
  - **Drill 3:** Numbers වලට `.toBe()` ද, Arrays වලට `.toEqual()` ද පාවිච්චි කරන්න.
- **Code Block:**

```
// Drill 1
test("adds numbers", () => {
  expect(2 + 2).toBe(4);
});
```

55. Guided Coding Task 1: formatName()

- **Concept Explanation (Sinhala):** `formatName` utility එක සහ ඒ සඳහා සම්පූර්ණ test suite එකක් නිර්මාණය කිරීම.
- **Code Block:**

```
// src/utils/formatName.js
export function formatName(str = "") {
  if (!str) return "";
  return str.charAt(0).toUpperCase() + str.slice(1);
}

// src/utils/formatName.test.js
import { describe, expect, test } from "vitest";
import { formatName } from "./formatName";

describe("formatName", () => {
  test("capitalizes lowercase input", () => {
    expect(formatName("react")).toBe("React");
  });

  test("returns empty string for empty input", () => {
    expect(formatName("")).toBe("");
  });

  test("returns empty string when no input is provided", () => {
    expect(formatName()).toBe("");
  });

  test("keeps already-capitalized input correct", () => {
    expect(formatName("React")).toBe("React");
  });
});
```

56. Guided Coding Task 2: Cart Total

- **Concept Explanation (Sinhala):** `calculateCartTotal` සඳහා tests ලියා පරීක්ෂා කිරීම.
- **Code Block:**

```
// src/utils/calculateCartTotal.js
export function calculateCartTotal(items = []) {
  return items.reduce((total, item) => {
    return total + item.price * item.quantity;
  }, 0);
}

// src/utils/calculateCartTotal.test.js
import { describe, expect, test } from "vitest";
import { calculateCartTotal } from "./calculateCartTotal";

describe("calculateCartTotal", () => {
  test("returns 0 for an empty cart", () => {
    expect(calculateCartTotal([])).toBe(0);
  });

  test("calculates total for one item", () => {
    expect(
      calculateCartTotal([{ price: 10, quantity: 2 }])
    ).toBe(20);
  });

  test("calculates total for multiple items", () => {
    expect(
      calculateCartTotal([
        { price: 10, quantity: 2 },
        { price: 5, quantity: 3 },
      ])
    ).toBe(35);
  });
});
```

57. Guided Coding Task 3: Button Component Test

- **Concept Explanation (Sinhala):** `Button` component එක render වෙනවද කියා user ගේ පැත්තෙන් පරීක්ෂා කිරීම.
- **Code Block:**

```
// src/components/Button.jsx
export default function Button({ label, onClick }) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}

// src/components/Button.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Button from "./Button";

test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);
  expect(screen.getByRole("button", { name: "Submit"
})).toBeInTheDocument();
});
```

58. Guided Coding Task 4: Alert Component Test

- **Concept Explanation (Sinhala):** message එකක් ඇති විට සහ නැති විට `Alert` component එක හැසිරෙන ආකාරය පරීක්ෂා කිරීම.
- **Code Block:**

```
// src/components/Alert.jsx
export default function Alert({ type = "info", message }) {
  if (!message) return null;
  return (
    <div role="alert">
      {type}: {message}
    </div>
  );
}

// src/components/Alert.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Alert from "./Alert";

test("shows an alert message", () => {
  render(<Alert type="error" message="Name is required" />);
  expect(screen.getByRole("alert")).toHaveTextContent(
    "error: Name is required"
  );
});

test("renders nothing when message is missing", () => {
  render(<Alert />);
  expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

59. Debug This Test (වැරදි ටෙස්ට් එකක් නිවැරදි කිරීම)

- **Concept Explanation (Sinhala):** - වැරදි කෝඩ් එක: `test("works", () => { expect(formatName("react")).toBe("react"); });`
  - ගැටලුව: Test name එක අපැහැදිලියි. බලාපොරොත්තු වන අගය වැරදියි (කැපිටල් විය යුතුය).
  - නිවැරදි කෝඩ් එක: පහත දැක්වේ.
- **Code Block:**

```
test("capitalizes lowercase input", () => {
  expect(formatName("react")).toBe("React");
});
```

60. Debug This Component Test (වැරදි Component ටෙස්ට් එකක් නිවැරදි කිරීම)

- **Concept Explanation (Sinhala):** - වැරදි කෝඩ් එක: `test("button", () => { render(<Button label="Submit" />); expect(screen.getByText("Cancel")).toBeInTheDocument(); });`
  - ගැටලුව: Test name එක අපැහැදිලියි, "Submit" label එක දීලා "Cancel" එකක් හොයනවා, props අඩුවෙන් දීලා තියෙනවා.
  - නිවැරදි කෝඩ් එක: පහත දැක්වේ.
- **Code Block:**

```
test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);
  expect(screen.getByRole("button", { name: "Submit"
})).toBeInTheDocument();
});
```

61. Build Something Small: Testing Mini Project (කුඩා ප්‍රොජෙක්ට් එකක් සාදන්න)

- **Concept Explanation (Sinhala):** ඉහත සකස් කළ `Button.jsx`, `Alert.jsx`, `formatName.js`, සහ `calculateCartTotal.js` යන ෆයිල් එකතු කරලා සම්පූර්ණ mini project එකක් හදලා ඒවට අදාළ tests රන් කරලා ඔක්කොම pass වෙනවද කියලා self-check එකක් කරන්න.

62. Challenge Task: Simple Signup Form Test (අභියෝගාත්මක පියවර)

- **Concept Explanation (Sinhala):** useState සහ සරල validation පාවිච්චි කරලා හදපු Form එකක් මුලින්ම නිවැරදිව render වෙනවද කියා බලන සරලම test එක මෙන්න.
- **Code Block:**

```
// src/components/SignupForm.jsx
import { useState } from "react";

export default function SignupForm() {
  const [name, setName] = useState("");
  const [submitted, setSubmitted] = useState(false);
  const nameIsValid = name.trim().length > 0;

  function handleSubmit(event) {
    event.preventDefault();
    setSubmitted(true);
    if (!nameIsValid) {
      return;
    }
    console.log("Submitted:", name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      {submitted && !nameIsValid && (
        <p role="alert">Name is required</p>
      )}
      <button type="submit">Sign up</button>
    </form>
  );
}

// src/components/SignupForm.test.jsx
import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import SignupForm from "./SignupForm";

test("renders name input and sign up button", () => {
  render(<SignupForm />);
  expect(screen.getByLabelText("Name")).toBeInTheDocument();
  expect(screen.getByRole("button", { name: "Sign up"
})).toBeInTheDocument();
});
```

63. Revision Questions & Answers (පුනරීක්ෂණ ප්‍රශ්න සහ පිළිතුරු)

- **Concept Explanation (Sinhala):** 1. **What is testing?** -> අපේ කෝඩ් එක බලාපොරොත්තු වන විදිහටම වැඩ කරනවද කියලා check කරන එක. 2. **Why is manual testing alone not enough?** -> App එක ලොකු වෙද්දී manual කරන එක හරිම slow, අමතක වෙන්න ලේසියි සහ එකම දේ නැවත නැවත කරන්න වෙනවා. 3. **What is automated testing?** -> වෙනත් කෝඩ් එකක් ලියලා අපේ කෝඩ් එකේ නිවැරදිභාවය පරීක්ෂා කිරීම. 4. **What does SDLC mean?** -> Software Development Life Cycle (මෘදුකාංගයක් හදනකොට පියවරෙන් පියවර යන සැලසුම් රටාව). 5. **What is a unit test?** -> කෝඩ් එකක තියෙන ඉතාම කුඩා කොටසක් (isolation එකක්) පරීක්ෂා කිරීම. 6. **What does test() do?** -> තනි test case එකක් ලියන්න පාවිච්චි කරයි. 7. **What does expect() do?** -> අපිට ලැබුණු සැබෑ ප්‍රතිඵලය (actual value) ඇතුළත් කරලා assertion එකක් පටන් ගනියි. 8. **When should you use toBe() and toEqual()?** -> සරල අගයන් වලට (numbers, strings) `toBe()`, objects සහ arrays වලට `toEqual()`. 9. **What is the TDD cycle?** -> Red (test එක ලියලා fail වෙනවා) -> Green (අවම කෝඩ් එක ලියලා pass කරගන්නවා) -> Refactor (කෝඩ් එක ලස්සන කරනවා). 10. **What is a mock, stub, and fake?** -> Mock: බොරු data. Stub: ස්ථාවර කෙටි උත්තරයක් දෙන function එක. Fake: ඇත්ත එක වගේම වැඩ කරන සරල එකක්. 11. **Why test visible behavior instead of implementation details?** -> User ට වැදගත් වෙන්නේ screen එකේ පේන දේ විතරයි. ඇතුලේ කෝඩ් එක ලියලා තියෙන විදිහ වෙනස් කරත් UI එක හරියට වැඩ කරයි නම් test එක pass විය යුතුයි.

65. Sprint 14 One-Page Memory Summary (එක් පිටුවක කෙටි මතක සටහන)

- **Concept Explanation (Sinhala):** - **Testing:** Expected behavior එක check කිරීම.
  - **SDLC:** Planning -> Design -> Implementation -> Testing -> Deployment -> Maintenance.
  - **Syntax:** `test("desc", () => { expect(actual).toBe(expected); })`
  - **Matchers:** `.toBe()` (values), `.toEqual()` (arrays/objects), `.toContain()` (array items), `.toBeInTheDocument()` (RTL presence).
  - **TDD:** Write test (Fail) -> Write code (Pass) -> Refactor.
  - **BDD:** Given -> When -> Then.
  - **React Testing Library:** User කෙනෙක් විදිහට UI එක පරීක්ෂා කරන්න උදව් වෙනවා (`render`, `screen`, `getByText`, `getByRole`, `queryByRole`).

66. Final Sprint 14 Mental Model (අවසන් මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** "මම දන්නවා මගේ කෝඩ් එකෙන් වෙන්න ඕනේ මොකක්ද කියලා → මම ඒ හැසිරීම විස්තර කරලා test එකක් ලියනවා → Test එක මඟින් කෝඩ් එක රන් කරනවා → සැබෑ උත්තරය සහ බලාපොරොත්තු වූ උත්තරය සංසන්දනය කරනවා → සමාන නම් Pass වෙනවා, නැත්නම් Fail වෙලා වැරැද්ද පෙන්වනවා." අපි දැන් අහන්න ඕනේ: "මගේ *app* එක දැකට විතරක් වැඩ කරනවද?" කියන එක නෙමෙයි, "මම කෝඩ් එක වෙනස් කරාට පස්සෙත් ඒ වැදගත් හැසිරීම් තවමත් හරියට වැඩ කරනවා කියලා මට ඔප්පු කරන්න පුළුවන්ද?" යන්නයි.

```
eof
```